



Fichas Técnicas de Estructuras Arbóreas y Aplicación Práctica

FICHA 1: Árbol Binario de Búsqueda (BST)

Definición: Árbol donde cada nodo tiene máximo 2 hijos. Los valores menores van a la izquierda y los mayores a la derecha.

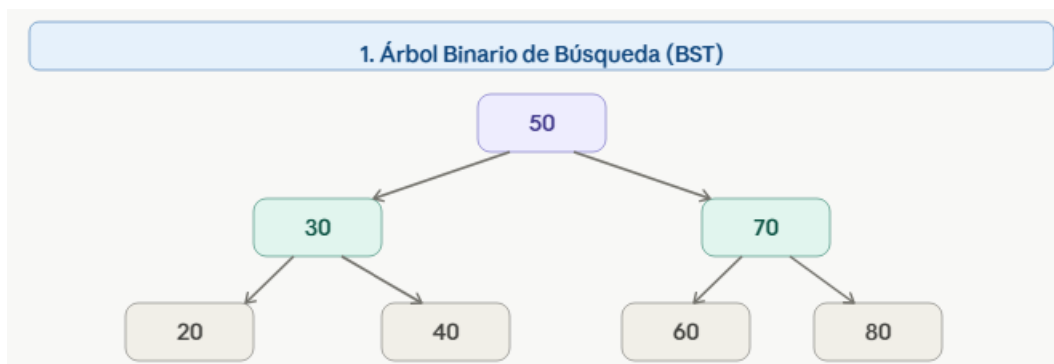
Operación	Caso Promedio	Peor Caso
Búsqueda	$O(\log n)$	$O(n)$
Inserción	$O(\log n)$	$O(n)$
Eliminación	$O(\log n)$	$O(n)$

Características Mecánicas: Se organiza comparando cada valor nuevo con el nodo actual y bajando izquierda o derecha según corresponda. No se balancea automáticamente.

Ventaja: Búsqueda rápida en datos aleatorios y estructura simple de implementar.

Limitación: Si los datos se insertan en orden ascendente o descendente, el árbol degenera en una lista lineal y pierde eficiencia.

Caso de Uso Real: Base conceptual de índices en bases de datos simples y sistemas de autocompletado básico.



FICHA 2: Árbol AVL

Definición: BST auto-balanceado que mantiene la diferencia de altura entre subárboles en máximo 1, usando rotaciones automáticas.

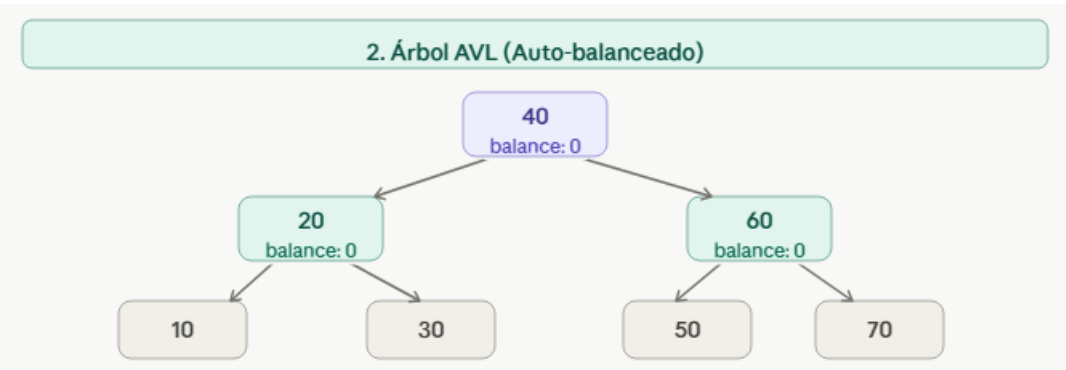
Operación	Caso Promedio	Peor Caso
Búsqueda	$O(\log n)$	$O(\log n)$
Inserción	$O(\log n)$	$O(\log n)$
Eliminación	$O(\log n)$	$O(\log n)$

Características Mecánicas: Cada nodo almacena un factor de balance (altura derecha - altura izquierda). Si ese factor sale del rango $[-1, 1]$, ejecuta rotaciones simples o dobles para restaurar el balance.

Ventaja: Garantiza $O(\log n)$ en todas las operaciones sin importar el orden de inserción.

Limitación: Las rotaciones frecuentes generan sobrecarga en aplicaciones con muchas inserciones y eliminaciones continuas.

Caso de Uso Real: TreeMap y TreeSet en Java, e índices internos de SQLite.



FICHA 3: Árbol B / B+

Definición: Árbol balanceado de múltiples hijos por nodo, diseñado para minimizar accesos a disco. El Árbol B+ almacena todos los datos en las hojas y las enlaza entre sí.

Operación	Caso Promedio	Peor Caso
Búsqueda	$O(\log n)$	$O(\log n)$
Inserción	$O(\log n)$	$O(\log n)$
Eliminación	$O(\log n)$	$O(\log n)$

Características Mecánicas: Cada nodo puede tener múltiples claves y múltiples hijos (factor de ramificación alto). Cuando un nodo se llena, se divide. Siempre permanece perfectamente balanceado en altura.

Ventaja: Extremadamente eficiente para lectura/escritura en disco por su alto factor de ramificación, que minimiza la cantidad de accesos necesarios.

Limitación: Implementación compleja. No es eficiente para datos en memoria RAM donde el acceso a disco no es el cuello de botella.

Caso de Uso Real: Motor de almacenamiento InnoDB de MySQL y sistemas de archivos como NTFS y ext4.



FICHA 4: Trie (Árbol de Prefijos)

Definición: Árbol donde cada nodo representa un carácter. Los caminos desde la raíz hasta las hojas forman palabras o cadenas de texto completas.

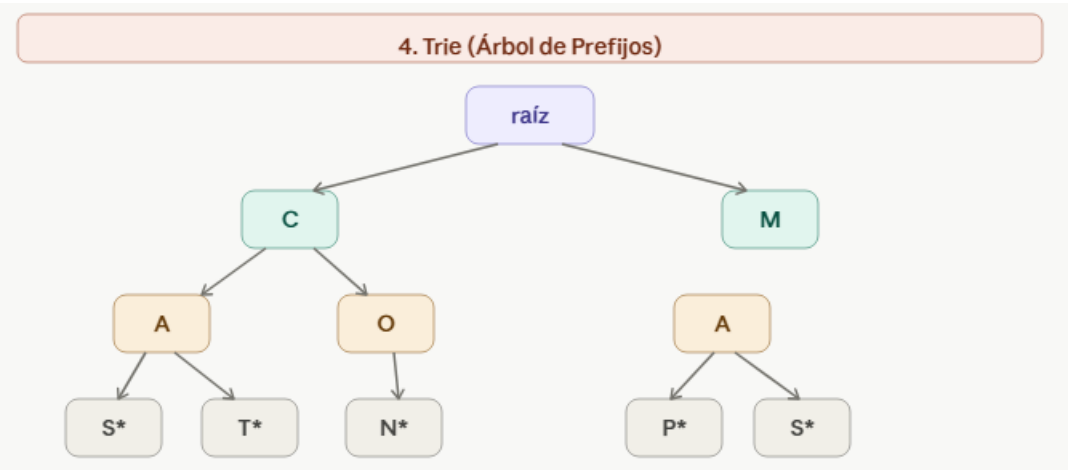
Operación	Caso Promedio	Peor Caso
Búsqueda	$O(m)$	$O(m)$
Inserción	$O(m)$	$O(m)$
Eliminación	$O(m)$	$O(m)$

Características Mecánicas: Cada nodo tiene hasta 26 hijos (uno por letra del alfabeto). Las palabras comparten prefijos comunes, ahorrando espacio. Un marcador especial indica el fin de una palabra válida.

Ventaja: Búsqueda por prefijo extremadamente rápida e independiente del número total de palabras almacenadas.

Limitación: Consume mucha memoria RAM, especialmente con alfabetos grandes o palabras con pocos prefijos compartidos.

Caso de Uso Real: Motor de autocompletado de Google Search y correctores ortográficos en procesadores de texto.



FICHA 5: Heap (Montículo Min/Max)

Definición: Árbol binario completo donde el nodo raíz siempre contiene el valor mínimo (Min-Heap) o máximo (Max-Heap) de toda la estructura.

Operación	Caso Promedio	Peor Caso
Búsqueda del mín/máx	$O(1)$	$O(1)$
Inserción	$O(\log n)$	$O(\log n)$
Eliminación del mín/máx	$O(\log n)$	$O(\log n)$

Características Mecánicas: Se implementa como un arreglo. Al insertar, el nuevo elemento se coloca al final y "sube" comparándose con su padre hasta encontrar su posición correcta (operación llamada sift-up). Al eliminar la raíz, el último elemento toma su lugar y "baja" (sift-down).

Ventaja: Acceso al mínimo o máximo en $O(1)$, ideal para sistemas que necesitan procesar siempre el elemento más prioritario.

Limitación: No sirve para búsquedas arbitrarias de valores que no sean el mínimo o máximo; esas operaciones son $O(n)$.

Caso de Uso Real: Algoritmo de Dijkstra para rutas más cortas (usado en Google Maps), colas de prioridad en sistemas operativos y schedulers de tareas.

